MODULE 1

Q1. Explain the Five Generic Framework Activities of a Software Process

A software process framework defines the overarching structure for software development. It comprises a set of framework activities populated by software engineering actions, which in turn are defined by task sets, all wrapped by umbrella activities (like quality assurance, project tracking, and configuration management).

The five generic framework activities are:

- **1. Communication:** Involves collaborating with the customer to understand stakeholders' objectives and gather requirements that define software features and functions.
- **2. Planning:** Creates a "software project plan" a map describing the technical tasks, likely risks, required resources, work products, and work schedule.
- **3. Modeling:** Involves the creation of models to help developers and customers better understand requirements and the software design.
- **4. Construction:** Combines code generation and the testing needed to uncover errors in the code.
- **5. Deployment:** The software is delivered to the customer, who evaluates it and provides feedback

Note: These five activities apply equally to small, simple programs, large web applications, and complex computer-based systems.

Q2. Explain Prescriptive Process Models (Waterfall Model, Incremental Model, and Spiral Model)

1. The Waterfall Model

- **Overview:** The waterfall model is the oldest paradigm for software engineering, proposing a systematic, sequential flow.
- **Workflow:** It moves strictly through the core framework activities: Communication -> Planning -> Modeling-> Construction -> Deployment.
- **Key Challenges:** Real projects rarely follow a strict linear flow; changes cause confusion as the project proceeds. It is difficult for customers to state all requirements explicitly at the start, and working versions of the program are only available late in the project timeline.
- **Suitability:** Best used when requirements are well-defined, stable, and new-development efforts are limited.

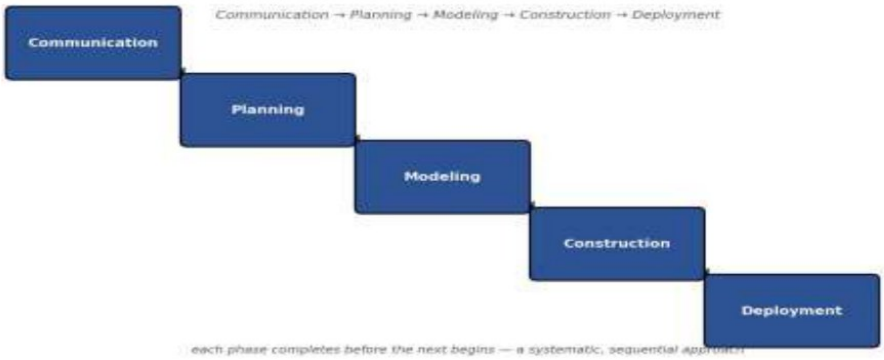
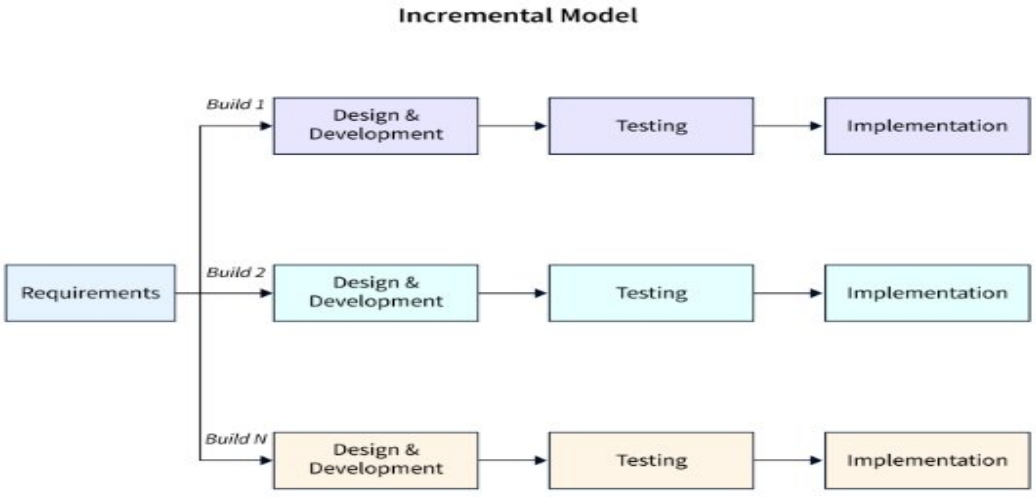


Fig 2.3 — The Waterfall Model: Communication → Planning → Modeling → Construction → Deployment.

2. The Incremental Model

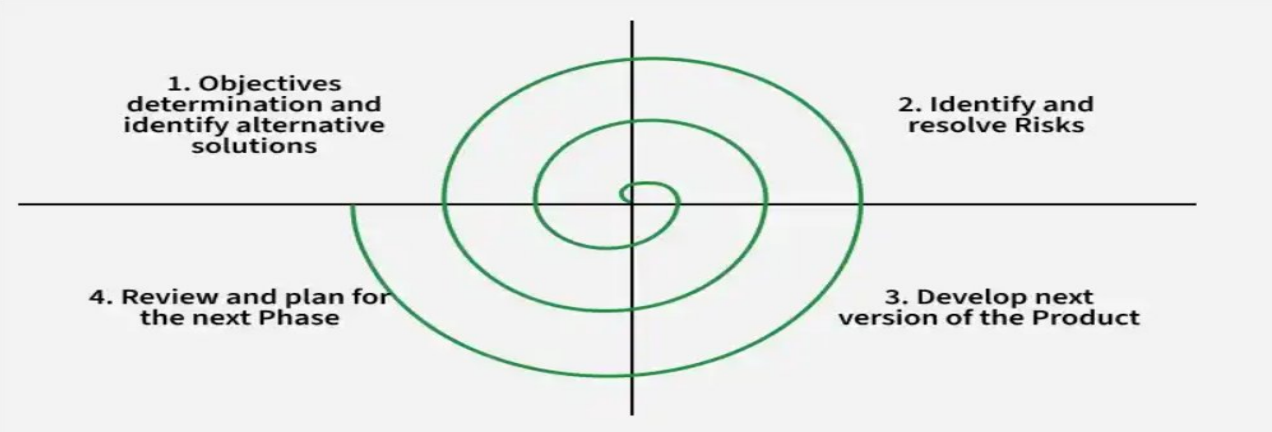
- **Overview:** The incremental model delivers a series of releases called "increments," which provide progressively more functionality to the customer with each delivery.
- **Workflow:** It combines elements of linear and parallel process flows. Each linear sequence produces a deliverable increment. For example, a word processor might deliver basic file management in increment 1, editing in increment 2, and spell-checking in increment 3.
- **Key Strategy:** The first increment is often a core product addressing basic requirements. Subsequent increments modify the core product and add features based on customer evaluation. It helps manage technical risk and accommodates staff shortages by staggering development.



3. The Spiral Model

- **Overview:** Originally proposed by Barry Boehm, the spiral model is an evolutionary software process model that couples the iterative nature of prototyping with the systematic, controlled aspects of the waterfall model.
- **Workflow:** It is a risk-driven process model structured as a cyclic approach. Starting at the center, each loop of the spiral passes through framework activities (Communication -> Planning -> Modeling -> Construction/Deployment), spiraling outward with increasing system completeness.

- **Key Features:** Risk is evaluated at every revolution using anchor-point milestones. The first circuit might result in a product specification, while later circuits build prototypes and fully functional software versions.



Q3. Concept of software application domains to classify given software system

software is applied in virtually every imaginable area, and over the years, certain application domains have evolved. To properly classify and understand a given software system, software engineering recognizes **seven broad categories** of software application domains:

- **1. System Software:** Programs that service other programs. Examples include compilers, editors, file-management utilities, operating system components, drivers, and networking/telecom software. They are characterized by heavy hardware interaction, multi-user usage, scheduling, and resource sharing.
- **2. Application Software:** Stand-alone programs that solve a specific business need. They often control business functions in real time, such as point-of-sale processing or manufacturing process control.
- **3. Engineering/Scientific Software:** Software characterized by "number-crunching" algorithms. Fields include astronomy, volcanology, automotive stress analysis, orbital dynamics, molecular biology, Computer-Aided Design (CAD), and system simulation.
- **4. Embedded Software:** Software that resides within a product or system to implement and control features for the end user and the system itself. Examples include microwave keypad control, automotive fuel control, and braking systems.
- **5. Product-Line Software:** Software designed to provide a specific capability for many different customers. It can target a limited/esoteric marketplace (such as inventory control) or a mass market (such as word processors, spreadsheets, graphics, entertainment, DBMS, and financial apps).
- **6. Web Applications ("WebApps"):** A network-centric software category ranging from simple linked hypertext files to sophisticated computing tools integrated with corporate databases and business applications.
- **7. Artificial-Intelligence (AI) Software:** Software that uses non-numerical algorithms to solve complex problems not amenable to direct computation or analysis. Examples include robotics, expert systems, pattern recognition, neural networks, theorem proving, and game playing.

MODULE 2

Q4. Use Case Diagram for the 'SafeHome' Security System

A use case diagram is a behavioral UML diagram that visualizes the interactions between external actors and the system's internal use cases. For the **SafeHome security system**, the use cases capture functional requirements from the perspective of the homeowner and monitoring services.

Actors and Use Cases Identified in the Module

- **Actors:**
  - **Homeowner:** The primary user/actor who interacts locally or remotely with the security control panel.
  - **Monitoring Service:** An external actor/system notified automatically when a security breach or emergency occurs.
- **Core Use Cases (Functions):**
  1. **Configure Security System:** Setting up passwords, sensor zones, and operational parameters.
  2. **Arm/Disarm System:** Activating or deactivating the home security network via keypad or remote triggers.
  3. **View Camera Surveillance:** Accessing live video feeds of home premises.
  4. **Account for Sensor Event:** Handling signals from door/window contacts, smoke detectors, or motion sensors.
  5. **Access Security System via Internet:** Remote login and monitoring through web interfaces or mobile apps.
  6. **Notify Monitoring Service:** Automated dispatch of emergency alerts to external security agencies.

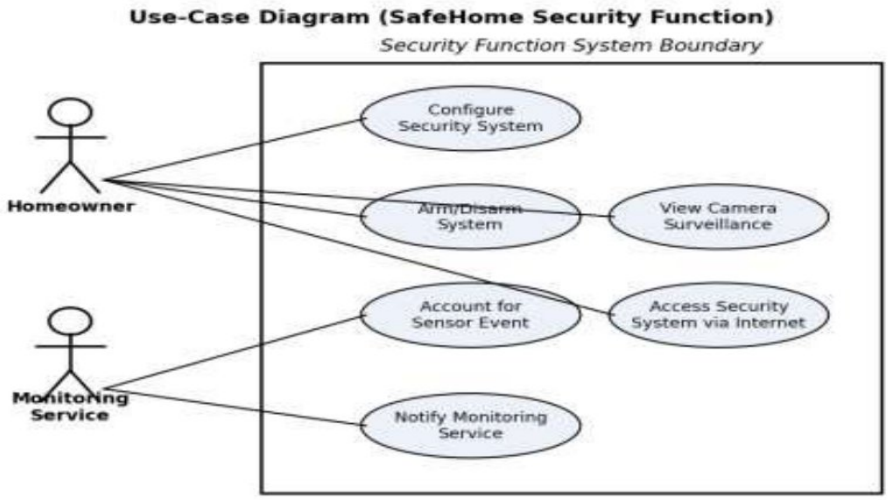


Fig. 6.1 – A use-case diagram showing actors and the use cases of the security-monitoring system (illustrative example)

Q5. Explain the Requirements Engineering Process and its Seven Distinct Tasks

Requirements Engineering (RE) is the foundational phase of software development. It encompasses all activities required to create and maintain a comprehensive Software Requirements Specification (SRS) document. Effective RE ensures that developers build the *right* system that satisfies stakeholder expectations.

The requirements engineering process comprises **seven distinct tasks**, as identified by Pressman:

1. **Inception**
    - **Description:** Stakeholders establish the basic problem, identify the people who want a solution, define the nature of the desired solution, and determine how the software and system fit into broader business or customer needs.
2. **Elicitation**
    - **Description:** Involves finding out from stakeholders what the objectives for the system are, what is to be accomplished, and how the software fits into existing business processes.
3. **Elaboration**
    - **Description:** The information gathered during inception and elicitation is expanded and refined. An analysis model is created that identifies specific data, function, and behavioural requirements.
4. **Negotiation**
    - **Description:** Requirements are prioritised, technical and project risks are identified, and rough estimates of resources are agreed upon with stakeholders through the reconciliation of conflicts.
5. **Specification**
    - **Description:** Requirements are represented in a formal, reviewable format—which can be a written document, a set of graphical models, a prototype, or any combination of these.
6. **Validation**
    - **Description:** The work products of requirements engineering are thoroughly examined for consistency, omissions, and ambiguity to ensure high quality.
7. **Requirements Management**
    - **Description:** A continuous set of activities that helps the software team identify, control, and track requirements and any subsequent changes as the project proceeds through its lifecycle.

Q6. Activities involved in establishing groundwork in software engineering

Establishing the groundwork is the very first step in the Requirements Engineering process. It lays the operational and collaborative foundation for the project by identifying key participants, bridging conflicting perspectives, and defining the initial problem boundaries before detailed elicitation begins.

The four key activities involved in establishing the groundwork are:

- **1. Identifying Stakeholders**
    - A stakeholder is anyone who benefits directly or indirectly from the system being developed (e.g., business operations managers, product managers, customers, end users, consultants, software engineers, and maintenance personnel).
    - Every stakeholder possesses a distinct view of the system along with unarticulated expectations.
- **2. Recognizing Multiple Viewpoints**
    - Because numerous stakeholders exist, requirements are examined from multiple points of view.

    - Each viewpoint frequently results in a different set of requirements, which can lead to conflicts. Collaboration is essential to reconcile these differences into a consistent set of requirements.
- **3. Working Toward Collaboration**
    - Requirements engineering is fundamentally a collaborative negotiation process.
    - Since stakeholder views may be incompatible, the goal is to encourage stakeholders to negotiate, play different roles, and converge toward a common, shared vision.
- **4. Asking the First Questions**
    - Analysts ask context-free questions focused on the customer, overall goals, and project benefits.

  Example: *'Who is behind the request for this work?'*, *'Who will use the solution?'*, *'What will be the economic benefit of a successful solution?'*, and *'Is there another source for the solution needed?'* These questions help the analyst understand the problem space while letting customers voice their perceptions.

Q7. Data Modeling Concepts

Data modeling is a crucial requirements analysis activity that structures and organizes data requirements. It provides a blueprint of how data is stored, processed, and interrelated within a software system before physical database implementation begins.

The core concepts of data modeling include:

- **Data Object:** A representation of composite information (such as a person, place, event, or description) that must be understood by the software. It encapsulates *only data* with no reference to operations. Examples include external entities, things, occurrences, roles, or structures.
- **Attributes:** Properties that define a data object. They name an instance, describe it, or reference another instance. At least one attribute must be defined as an identifier or **'key'** to find, sort, or uniquely identify an instance.
- **Relationships:** The connections established between data objects (e.g., between `Book` and `Bookstore`, connections include `'is stocked in'`, `'is ordered by'`).
- **Cardinality:** The specification of the number of occurrences of one object that can be related to the occurrences of another. It is usually expressed as **one-to-one (1:1), one-to-many (1:M), or many-to-many (M:N)**.
- **Modality:** The constraint indicating whether a relationship is mandatory or optional. Modality is **0** if the relationship is optional (no explicit need to occur) and **1** if the relationship occurrence is mandatory.
- **Entity-Relationship Diagram (ERD):** The graphical modeling tool used to represent data architecture. It utilizes rectangular boxes for entities, diamonds for relationships, and numerical annotations to represent cardinality and modality clearly.

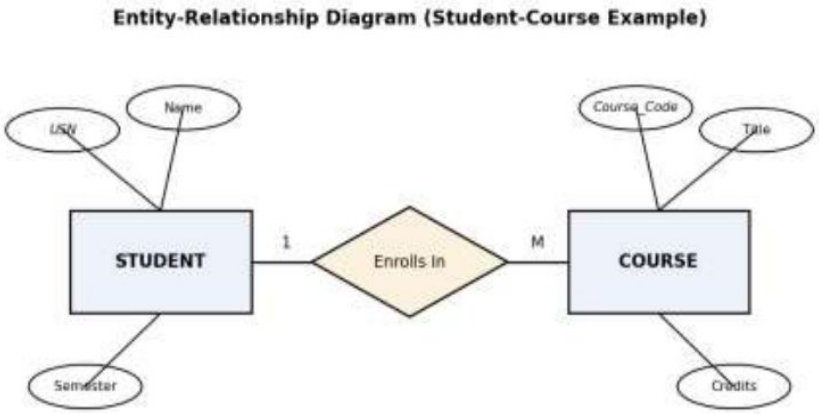


Fig. 6.2 – Entity-relationship diagram illustrating entities, a relationship, attributes, and cardinality

MODULE 3

Q8. Explain the Extreme Programming (XP) Process.

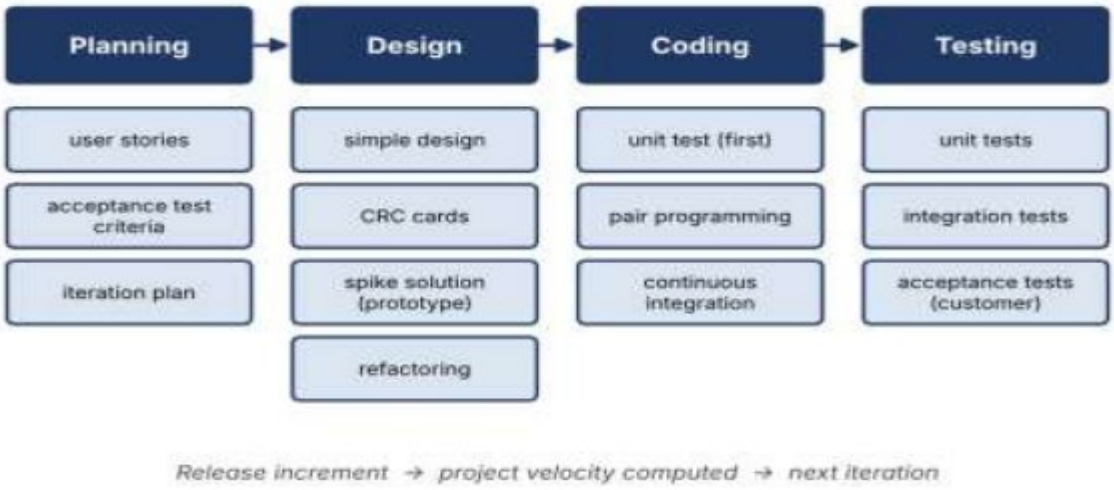
Extreme Programming is a lightweight, highly disciplined agile methodology designed to improve software quality and responsiveness to changing customer requirements through frequent releases in short development cycles.

The XP Process: Four Framework Activities

1. **Planning**
   - Begins with **listening** to understand the business context.
   - Leads to the creation of **user stories** written on index cards by the customer, who assigns a business priority value.
   - The team assigns a cost in development weeks; stories exceeding three weeks are split. Stories are grouped into software increments based on project **velocity** (number of stories implemented per release).
2. **Design**
   - Follows the KISS principle; provides implementation guidance for a story as written—nothing more, nothing less.
   - Utilizes **CRC cards** for object-oriented design thinking.
   - Uses **spike solutions** (operational prototypes) to lower risk when tackling difficult design problems.
   - Encourages **refactoring**—improving the internal structure of code without changing its external behavior.
3. **Coding**
   - Requires developers to write **unit tests** *before* writing the actual code.
   - Implements **pair programming**, where two developers work together at one workstation for real-time problem solving and quality assurance.
   - Uses **continuous integration** to merge code and perform smoke-testing early.
4. **Testing**
   - Unit tests are automated for easy regression testing whenever code changes.
   - Involves daily integration, validation, and system testing.

- **Acceptance tests** (customer tests) are specified by the customer to verify overall visible system features.

Fig. 2 The Extreme Programming (XP) process



Q9. Agile process development

Agility is the ability of a software team to respond effectively to change. An agile process treats change as normal and builds the development process around it rather than resisting it.

Any agile software process is characterized by the way it addresses three key assumptions about software projects:

- **Unpredictability of Requirements:** It is difficult to predict in advance which software requirements will persist and which will change, or how customer priorities will shift.
- **Interleaving of Design and Construction:** For many types of software, design and construction are performed together so that design models are proven as they are created.
- **Unpredictability of Workflow:** Analysis, design, construction, and testing are not as predictable from a planning point of view as traditional models assume.

To manage unpredictability and embrace change, agile processes incorporate the following mechanisms:

- **Incremental Adaptation:** Continual adaptation is achieved by securing customer feedback through operational prototypes or small increments of working software delivered in short cycles.
- **People and Interaction Focus:** Valuing individuals and team interaction over rigid processes, tools, and heavy documentation.
- **Active Customer Participation:** Drawing the customer directly onto the team so they help steer project priorities.
- **Flattening the Cost of Change:** Unlike conventional models where change costs rise non-linear over time, agile practices (such as continuous integration, test-driven development, and small releases) aim to keep change cheap throughout the project lifecycle.

Fig. 1 Change costs as a function of time in development

